

CORE: When Do Latent Representation Edits Behave Like Causal Interventions?

Anonymous ACL submission

Abstract

Latent representation edits are often described as causal interventions when they produce a desired answer, but output correctness alone does not show that an edit changed the intended variable, preserved unrelated information, or behaved consistently under repeated and composed commands. We evaluate latent editors using three separate evidence gates: task correctness, structural route diagnostics, and non-trivial controls. Across four decoder backbones and 23 registered analyses, supervision reduced identity, idempotence, and disjoint commutation residuals, but four-law training produced a small decrease in balanced accuracy relative to one-law training of -0.00356 , with a 95% confidence interval of $[-0.00513, -0.00199]$. The simpler instruction-conditioned *O2* editor outperformed the proposed state-aware *O3* editor on all four matched backbone means, while wrong target and inverted value controls changed downstream decisions very little across three backbones. Same target overwrite sequence accuracy remained between 3.42 and 7.18%. Furthermore, do nothing and randomly initialized editors achieved zero route residuals, showing that algebraic agreement can be satisfied vacuously. Selective invariance also showed no measurable benefit under its dedicated objective. We therefore treat the laws as diagnostics rather than certificates of causality, as a latent edit warrants an intervention claim only when it makes the requested change, preserves the appropriate context, responds to causal controls, and outperforms trivial alternatives.

1 Introduction

The world is understood through language, and with the adoption of artificial intelligence through large language models (LLMs), storytelling has become a distinct way to craft a world simulation for these models (Gurnee and Tegmark, 2024; Gkoutouras et al., 2025). For instance, in a small world

where it is summer, the gardener runs the sprinkler, and it does not rain. Either the sprinkler or rain makes the grass wet. If a model is asked, it could correctly answer that the grass is wet.

Our main goal is to ask what happens under the operator `do(sprinkler = off)`. Indeed, there are many ways to obtain the answer “dry,” and we can start by intervening in the prompt style (Liu et al., 2023, 2026b), fine-tuning the model’s parameters (Ding et al., 2023; Wang et al., 2025), adding a steering vector (Zou et al., 2023; Stolfo et al., 2025), or editing a hidden representation in the model (Zou et al., 2023; Gu et al., 2026). However, a correct answer alone does not show that the edit changed the representation associated with the named target or produced the downstream and preservation effects implied by the task graph (Yeo et al., 2025). We test this operationally with target/value perturbations, descendant-change probes, non-descendant-preservation probes, and edit routes. Therefore, we want to determine if intervening on the internal features of an LLM can more reliably induce the intended causal change in its outputs (Geiger et al., 2021; Wu et al., 2023).

To achieve this, we use four diagnostic laws. The first is identity (Pearl et al., 2003), which evaluates whether intervening on a model’s variable by setting it to its current value leaves its representation unchanged. The second is idempotence (Pearl et al., 2003; Geiger et al., 2021), which tests whether applying the same intervention twice produces the same result as applying it once. The next is disjoint commutation (Rubenstein et al., 2017; Kolbeinson et al., 2025), which tests whether interventions on distinct and non-overlapping variables produce the same result regardless of the order in which they are applied. Finally, selective invariance tests whether the intervention changes only the targeted causal variable without changing any other unrelated aspects of the represented world (Bing et al., 2023; Bongers et al., 2021).

Source	Language/structure	Intervention type	What it provides	Role in CORE
XOR synthetic	templated, full graph	binary value-set	exact structural equations and complete ground-truth states	controlled operator replication, law training, floors
CCR.GB (Maasch et al., 2025)	generated, full SCM	value-set	multiple SCM structures with explicit causal dependencies	second graph family; graph-derived descendants/non-descendants
CLadder (Jin et al., 2023)	templated/verbalized graph	intervention / counterfactual	causal questions derived from explicit DAGs and SCMs	cross-family replication, estimand diagnostic
WIQA (Tandon et al., 2019)	human procedural prose, signed graph	qualitative more/less/no-effect	qualitative influence relations over procedural events	natural-language transfer and preservation under qualitative effects
Com ² (Xiong et al., 2025)	event chains	event replacement	commonsense event sequences with alternative event substitutions	open-vocabulary event editing; weaker preservation structure
CounterBench (Chen et al., 2026) / CRASS	held-out causal tasks	external evaluation	unseen causal reasoning problems outside the training families	T5 transfer evaluation
Evidence Inference 2.0 (DeYoung et al., 2020)	biomedical evidence text	lower-rung prediction	intervention-outcome relations from biomedical study reports	diagnostic for retention of lower-rung causal competence

Table 1: Data sources are used only for intervention properties supported by their available structure.

A do-nothing operator, for example, can trivially satisfy laws such as idempotence and disjoint commutation because repeatedly making no change, or changing nothing in different orders, would naturally produce the same result (Hewitt and Liang, 2019). Consider a light switch that controls a bulb and a separate faucet that controls the flow of water. If an intervention operator does nothing when asked to turn the light switch off and does nothing again when the same intervention is repeated, both operations will produce the same result, apparently satisfying idempotence. Similarly, asking it to turn off the light and close the faucet in either order will also produce the same unchanged world, apparently satisfying disjoint commutation. Yet, neither intervention was actually performed. However, this seemingly perfect operator would fail when an intervention is actually expected to change the represented world. Similarly, a random editor that effectively forgets the underlying representation could still produce perfect route agreement without preserving the intended causal structure (Rubenstein et al., 2017; Gu et al., 2026).

The second is that an operator can score well on the tests without performing the actual task. We tested a randomly initialized operator, and it successfully passed 100% of the mathematical consistency tests. However, its accuracy on the actual question was only 52.32%, which was near random chance.

Our evaluation therefore checks three things together: task accuracy (did it get the answer right? Structural rules (how much did it obey the logic?), and non-trivial controls (did it actually change the represented world, or did it rely on shortcuts such as doing nothing? This paper asks one overarching question: When does a latent representation edit behave like a causal intervention rather than merely producing a useful change in an answer?

RQ1 Can explicit supervision teach a latent editor the structural properties associated with interventions without sacrificing task performance? **RQ2** Do the learned editors control the intended part of the represented world, and is the claimed addressing pathway responsible for the resulting decision? **RQ3** Do learned edits compose across variables and correctly overwrite an earlier value when the same target is edited again? **RQ4** Does the learned behavior survive paraphrases, unseen causal environments, changes in causal reasoning tasks, external benchmarks, and biomedical evidence? Our work makes five main contributions.

- First, we built a standardized benchmark and dataset with a four-step audit process for testing whether intervention operators are genuine or otherwise.
- Second, we trained AI models with custom loss objectives that force them to obey one, three, or all four rules. Although this made their internal operations mathematically cleaner, it did not produce a statistically significant positive effect on basic task performance.
- Third, we tested a wide range of prompting and editing methodologies and found that several supposedly different methods were using identical underlying weights. We also found that while pointing tools could locate where to edit the model, changing those targets did not always control the final decision.
- Fourth, across 4 main experiments and 4 models, we introduce an advanced operator, O3. O3 helped the models transfer their reasoning to new, outside causal tasks, but struggled when information had to be overwritten multiple times in a row.
- Finally, we are releasing our dataset to the public, including exactly 900 source examples independently annotated by two human reviewers, with

all disagreements resolved.
For the 900 rows (300 from CausalT5K, 300 from METER, and 300 from PubMedCausal), the annotators had a 98.58% observed agreement with multiclass Cohen’s kappa = 0.5092. A senior reviewer resolved the 17 decision disagreements.

2 Data Construction and Models

Testing causal laws requires richer metadata than standard question-answering datasets (Jin et al., 2023), so every benchmark item is structured around a unified schema containing (1) the baseline context, (2) the intervention command, (3) the target outcome, (4) causal dependencies, i.e., descendants and non-descendants (Pearl et al., 2003), introduced by a graph, and (5) targeted verification probes that identify invariance.

Free-text intervention experiments additionally record the exact target span in the text, a compact value token, and the natural-language intervention text. Any missing structural information is left explicitly undefined. *Example:* In a lawn-care context with initial state {season = summer, sprinkler = on, rain = off, wet = yes}, we apply the intervention do(sprinkler = off). Ground wetness is a descendant that should update to “no,” whereas the season is an independent non-descendant that must remain “summer.” A single benchmark item can thus simultaneously evaluate target execution, downstream impact, non-target preservation, and algebraic route laws. The datasets used are discussed in Table 1. We use four decoder LLMs: Qwen2.5-7B, Phi-4-14B, Qwen2.5-32B, and Llama-3.3-70B. The design choice is motivated by (1) within-family scale (Qwen 7B to 32B) and (2) cross-family architectures.

3 CORE Methodology

3.1 Intervention Semantics and Evaluation

Let h be a hidden representation of a described world and e an intervention instruction. An editor defines

$$T_e(h) = h + \Delta_\theta(h, e). \quad (1)$$

If T_e is interpreted as implementing do($X = x$), we test the following:

- Identity (?): The empty intervention should preserve the state: $T_{e_\emptyset}(h) \approx h$.
- Idempotence: Repeating the same hard intervention should not keep moving the state (e.g., turning the sprinkler off twice should equal turning it

off once):

$$T_e(T_e(h)) \approx T_e(h). \quad (2)$$

- Disjoint commutation: Changing two unrelated variables should not depend on application order. For appropriately disjoint targets a, b ,

$$T_b(T_a(h)) \approx T_a(T_b(h)). \quad (3)$$

- Selective invariance: If the intervention targets X , non-descendants of X should remain stable. Turning the sprinkler off may change wet grass, but must not change season. We use behavioral probes generally and representational measurements only when variable localization is defensible.

Due to additive laws, testing the operator on just the output fails, because simple addition works in any order. We therefore test a discriminative case: $r_1 : X \leftarrow 0$ followed by $X \leftarrow 1$ should agree with the state produced by $r_2 : X \leftarrow 1$ alone (Pearl et al., 2003), revealing whether the AI editor genuinely overwrites old information or just keeps piling on changes on top of each other. To measure this agreement, let a and b denote the final representations produced by the two routes being compared. We use the normalized squared distance

$$d(a, b) = \frac{\|a - b\|_2^2}{\|a\|_2^2 + \|b\|_2^2 + \epsilon} \begin{cases} 0 & \text{identical,} \\ 1 & \text{unrelated,} \\ 2 & \text{opposite} \end{cases} \quad (4)$$

Using Cauchy–Schwarz, $0 \leq d < 2$: $d = 0$ is exact route agreement of a, b . For an exact oracle that maps both equivalent intervention routes to the same ground-truth intervened representation, the theoretical route residual is 0. Before testing the LLMs, we verified the framework on a simpler, solved system using XOR logic and the CCR.GB ground-truth benchmark. Across the test runs (600 identity, 1,800 idempotence, and 2,974 commutation evaluations on CCR.GB), the framework returned an exact zero error score, showing that it correctly recognizes when the expected mathematical rules are satisfied. However, machine learning evaluations often rely on arbitrary cutoffs (e.g., if threshold ≥ 0.5 , the method works) or measures such as cosine similarity to determine whether generated text matches the actual text. In our use case, such a score alone is not sufficient because a do-nothing operator can also achieve a perfect zero error score. Rather than deciding that an operator

works based on an arbitrary cutoff, our framework uses a baseline comparison mode. It directly compares the operator against three trivial baselines: doing nothing, doing everything, and acting randomly, alongside its actual task accuracy. An operator must therefore be evaluated not simply by whether its score is low or high, but by whether it performs better than these trivial alternatives while actually producing the intended change (Liu et al., 2026a).

To prevent models from superficial compliance via “do-nothing” or indiscriminate operations, we score interventions using a balanced average of change accuracy, which measures whether variables that should change are changed correctly, and preservation accuracy, which measures whether non-descendants remain factually accurate (Meng et al., 2022). Since $L1/L2/L3$ datasets do not capture a harmonized change, we cannot generalize selective-invariance to these particular experiments.

3.2 Latent operator family

Let the hidden representation at layer ℓ of an LLM processing text x be $h_\ell = F_{1:\ell}(x)$, where h_ℓ is the intermediate hidden state, and $F_{1:\ell}(x)$ represents the processing executed by the first ℓ layers of the model in question. To perform an intervention using description e and an optional structural information G , we keep the model’s weights constant and apply the editing operator $h'_\ell = T_e(h_\ell)$ to tweak the hidden state, allowing the remaining transformer layers to complete processing. We compare three designs for this editor tool:

- **O1 (Fixed-Vector Edit):** Adds a fixed direction vector for each intervention type. It serves as a standard steering baseline (Turner et al., 2024), but because it blindly adds the same non-zero vector every time, repeated applications cannot satisfy exact idempotence.
- **O2 (Instruction-Conditioned Edit):** Generates an update based purely on the command text e , without reading the current hidden state. This mirrors standard language-conditioned steering (Subramani et al., 2022).
- **O3 (State-Aware Edit):** Reads both the command instruction e and the AI’s current hidden state h_ℓ .

$$\Delta_\theta(h, e) = g_\theta(h, e) \odot U_\theta \sigma(V_\theta[h; e]). \quad (5)$$

Here, $[h; e]$ concatenates the current hidden state and instruction, V_θ and U_θ form a low-rank transformation, and $g_\theta(h, e)$ acts as a gate that can scale

down or zero out the update if the requested state is already active. This enables the editor to recognize whether a command has already been carried out.

3.2.1 Operator Localization

For each experiment, the underlying model uses fixed parameters while the editor updates the representation. The editor applies its modification once at a designated intermediate layer, and the remaining transformer blocks process the edited state to generate the final output.

3.3 Addressing Free-Text Targets

Synthetic datasets have fixed labels, which are not representative of real-world text, which is unpredictable. For instance, an intervention may target a sprinkler, as in our case, or a more complex event, such as a medical condition. It is therefore the job of our target-finding method to determine where that target concept is represented within the LLM’s internal state. We compare four different target-finding methods:

1. T0 (Fixed Lookup), uses a preset lookup table for artificial variables and serves as a theoretical upper-limit baseline.
2. T1 (Trainable Text Encoder) uses a dedicated text processor trained to map the intervention sentence directly into the edit condition.
3. T2 (Masked In-Context Insertion) appends the intervention command directly to the text prompt using attention masking, so the LLM does not absorb the instruction prematurely, and the instruction can be erased after the edit occurs.
4. T3 (Pointer-Based Target Matching) scans the LLM’s internal hidden states using an attention mechanism to point directly to the target phrase.

$$q = W_q a, \quad k_i = W_k h_i, \quad (6)$$

$$\pi_i = \text{softmax}(q^\top k_i / \sqrt{d}), \quad \Delta_i = \pi_i O(h_i, [a; v]). \quad (7)$$

For pointer-based target matching, we turn the target phrase into a query (Q), compare it against key vectors (K_i) across the internal states, generate a probability score (π_i), and apply the edit (Δ_i) based on that match. Even if a pointer tool scores 100% accuracy at locating the target phrase, finding

the right location does not equate to controlling the final answer. Randomizing the pointer should also affect the downstream behavior.

3.4 Nested law objectives

To test how much reinforcement is required, we evaluate three nested levels of rule enforcement:

- CORE-Base (1 law): Enforces selective invariance
- CORE-I (3 laws): Adds identity and idempotence
- CORE-Full (4 laws): Adds disjoint commutation

The overall training objective is

$$\mathcal{L} = \mathcal{L}_{task} + \lambda_m \mathcal{L}_{match} + \lambda_{inv} \mathcal{L}_{inv} + \lambda_{id} \mathcal{L}_{id} + \lambda_{idem} \mathcal{L}_{idem} + \lambda_{comm} \mathcal{L}_{comm} + \lambda_{cf} \mathcal{L}_{cf}. \quad (8)$$

$\mathcal{L}_{match}$ ensures the model completes its primary task, matches the ground-truth target state, and generates logically consistent counterfactual answers.

Algebraic Rule Terms ($\mathcal{L}_{inv}$, $\mathcal{L}_{id}$, $\mathcal{L}_{idem}$, $\mathcal{L}_{comm}$) measure distances between internal hidden-state routes to penalize violations of selective invariance, identity, idempotence, and commutation. Unrelated variables are explicitly protected from unintended effects. Exact executable definitions and masking conventions are given in Appendix A.

4 Study Design

The study contains a validation stage, which is used for model development and analysis, followed by four experiments. The validation stage uses systems in which the correct intervened state is known, checking whether the evaluation gives zero residuals when an intervention is correct and if it can identify operators that do nothing or act randomly.

Experiment 1 addresses RQ1 by comparing one-law, three-law, and four-law training and by removing each objective in turn. Experiment 2 addresses RQ2 by comparing operators, removing the learned intervention, changing the intervention target and value, and testing placebo and shuffled conditions. Experiment 3 addresses RQ3 through combined interventions and repeated interventions on the same target. Experiment 4 addresses RQ4 through paraphrases, unseen causal environments, causal-ladder tasks, external benchmarks, and biomedical evidence.

4.1 Validation Results

The validation asks whether the evaluation recognizes a correct intervention and whether it rejects

trivial solutions. On XOR and CCR.GB, identity, idempotence, and commutation residuals are exactly zero while task correctness is 1.000. This shows that the evaluation returns zero residuals when the intervention is known to be correct; however, zero residual is not enough. In the language-model setting, do-nothing and randomly initialized operators also reach zero residual while balanced task accuracy remains near 0.534 to 0.535. A randomly initialized operator passed the mathematical consistency tests while reaching only 53.41% task accuracy. Doing everything produces high structural residuals and only modest task accuracy. We therefore evaluate the mathematical rules together with task correctness and control for trivial solutions.

5 Experiments

5.1 Experiment 1: Learning the Intervention Laws

5.1.1 One-Law, Three-Law, and Four-Law Training

Table 2 shows the macro-average results of the four models. Across the four models, one-law, three-law, and four-law training yield nearly identical balanced task accuracy: 0.6220, 0.6222, and 0.6185, respectively, and this holds across models.

Relative to one-law training, four-law training changes balanced accuracy by only -0.0036 [95% CI -0.0051, -0.0020]. It reduces idempotence residual by -0.5501 [-0.5970, -0.5031], commutation residual by -0.1413 [-0.1694, -0.1133], and last-write residual by -0.6279 [-0.6945, -0.5614]—identity residual falls by -0.00293 [-0.00340, -0.00247]. The intervention laws therefore have a much larger effect on the structural residuals than on task accuracy.

5.1.2 Law Ablation: Removing One Law at a Time

Ablation results are in Table 3, where we see that removing identity increases the identity residual by +0.1197 [0.1104, 0.1289]. Removing idempotence increases the idempotence residual by +0.1807 [0.1723, 0.1891]. Removing commutation increases the commutation residual by +0.1933 [0.1789, 0.2076]. Changes in balanced accuracy are very small.

Removing one law can also affect the other residuals. Removing commutation lowers the idempotence residual even while the commutation residual

Table 2: Laws Changes vs Task Accuracy

Training condition	Balanced accuracy	Identity residual	Idempotence residual	Commutation residual	Last-write residual
One-law training	0.6220 [0.6072, 0.6369]	0.0037 [0.0031, 0.0042]	0.6248 [0.5758, 0.6738]	0.1916 [0.1612, 0.2220]	0.7756 [0.7036, 0.8476]
Three-law training	0.6222 [0.6075, 0.6369]	0.0015 [0.0012, 0.0017]	0.1039 [0.0913, 0.1165]	0.0840 [0.0780, 0.0900]	0.2468 [0.2298, 0.2638]
Four-law training	0.6185 [0.6042, 0.6327]	0.0007 [0.0006, 0.0008]	0.0747 [0.0676, 0.0819]	0.0502 [0.0471, 0.0533]	0.1477 [0.1401, 0.1553]
Do nothing	0.5349 [0.5306, 0.5391]	0	0	0	0
Random initialization	0.5341 [0.5299, 0.5383]	0	0	0	0
Do everything	0.5466 [0.5390, 0.5543]	0.9982 [0.9938, 1.0027]	1.0000 [0.9995, 1.0006]	0	1.0000 [0.9995, 1.0006]

Table 3: The leave-one-law-out ablation tests whether each improvement is tied to its objective.

Condition	Balanced accuracy	Identity residual	Idempotence residual	Commutation residual	Last-write residual
Four-law training	0.6201 [0.6054, 0.6347]	0.0008 [0.0006, 0.0009]	0.0757 [0.0683, 0.0832]	0.0503 [0.0474, 0.0533]	0.1501 [0.1423, 0.1579]
Remove identity	0.6202 [0.6055, 0.6349]	0.1205 [0.1112, 0.1297]	0.0763 [0.0688, 0.0838]	0.0504 [0.0475, 0.0534]	0.1507 [0.1430, 0.1585]
Remove idempotence	0.6194 [0.6045, 0.6343]	0.0014 [0.0012, 0.0017]	0.2564 [0.2440, 0.2688]	0.0158 [0.0144, 0.0171]	0.2668 [0.2540, 0.2797]
Remove commutation	0.6227 [0.6077, 0.6376]	0.0007 [0.0006, 0.0009]	0.0039 [0.0033, 0.0046]	0.2436 [0.2283, 0.2589]	0.2803 [0.2624, 0.2981]
Remove selective invariance	0.6235 [0.6085, 0.6386]	0.0015 [0.0012, 0.0018]	0.1071 [0.0936, 0.1206]	0.0842 [0.0786, 0.0898]	0.2526 [0.2350, 0.2703]

increases. Removing either idempotence or commutation also increases the last-write residual. The laws therefore affect their corresponding residuals, but their effects are not completely separate.

5.1.3 Selective Invariance

Selective invariance measures changes to descendants together with preservation of non-descendants. Across the models, the score remains similar across one-law, three-law, and four-law training. On Llama-3.3-70B, the means are 0.6225, 0.6210, and 0.6186, with overlapping intervals. Removing selective invariance gives 0.6205 compared with 0.6189 for the full objective. In the separate BERT-base analysis, removing selective invariance gives 0.49377 compared with 0.49383 on XOR and exactly 0.51356 compared with 0.51356 on CLadder. The results do not show that the current selective-invariance objective improves this measure. Identity, idempotence, and commutation improve with training, but selective invariance does not.

5.1.4 Evaluation Mechanisms

Our structured evaluation mechanism compares several intervention methods using the same measures as shown in Table 4. Parameter-efficient fine-tuning (PEFT), representation fine-tuning (RFT), and the instruction-conditioned operator (ICO/O2) are evaluated using the same models within each model and seed. The fixed-vector and task-vector methods also use the same evaluated models. Prompting has zero residuals because it does not change the internal representation, while the state-aware operator has higher task accuracy but larger residuals.

5.2 Experiment 2: Testing Control

Our second experiment focuses on understanding if the internal operators actually control the model’s output. Here, we focus on five sub-experiments.

5.2.1 Instruction-Conditioned and State-Aware Operators

The O3 operator uses the current hidden representation when deciding how much to change. Across various runs, the instruction-conditioned operator performs better on three of the four models and has a higher numerical result on all four, as three confidence intervals exclude zero, while the interval for Phi-4 includes zero. Using the current hidden representation does not improve performance across all four models, as seen in Table 5.

5.2.2 Operator Ablation: Removing the Learned Intervention

We remove the learned intervention while keeping the rest of the model unchanged. From Table 6, the active operator performs better than the zeroed and no-instruction conditions across all four models. This shows that the learned intervention contributes to the model’s answer.

5.2.3 Changing the Target and Intervention Value

The model identifies the recorded target correctly in the evaluated examples, but balanced intervention accuracy remains about 0.539 to 0.546. Randomizing the selected target changes performance very little on most models. We therefore also test a fixed wrong target and an inverted intervention value.

Phi-4 shows a decrease in target success for both changes. Qwen2.5-7B shows no change in its final

Method	Balanced accuracy	Identity residual	Idempotence residual	Commutation residual	Last-write residual
Prompting	0.5370 [0.5278, 0.5461]	0	0	0	0
PEFT	0.5024 [0.4838, 0.5209]	0.0400 [-0.0635, 0.1435]	0.0276 [-0.0420, 0.0973]	0.0202 [-0.0302, 0.0707]	0.0276 [-0.0420, 0.0971]
RFT	0.5075 [0.4823, 0.5327]	0.0022 [-0.0039, 0.0084]	0.0116 [-0.0165, 0.0398]	0.0074 [-0.0170, 0.0318]	0.0114 [-0.0163, 0.0391]
ICO/O2	0.5082 [0.4969, 0.5194]	0 [0, 0]	-0.0019 [-0.0070, 0.0031]	0.00002 [-0.00019, 0.00022]	-0.0008 [-0.0024, 0.0008]
Fixed-vector / task-vector	0.5438 [0.5289, 0.5586]	1.0040 [0.9936, 1.0143]	0.9919 [0.9853, 0.9985]	0	0.9919 [0.9853, 0.9985]
Router	0.5420 [0.5278, 0.5562]	0.9048 [0.7596, 1.0499]	0.8907 [0.7480, 1.0334]	0	0.8907 [0.7480, 1.0334]
State-aware operator /O3	0.5972 [0.5626, 0.6318]	0.7016 [0.5687, 0.8345]	0.7698 [0.6381, 0.9015]	0.1954 [0.1327, 0.2580]	0.9198 [0.7507, 1.0888]

Table 4: Comparison of balanced accuracy and algebraic residuals across editing methods. Values are reported with 95% confidence intervals.

Model	O2-O3	95% CI	O2 mean	O3 mean
Qwen2.5-7B	+0.0277	[0.0207, 0.0348]	0.7705	0.7427
Phi-4-14B	+0.0045	[-0.0001, 0.0091]	0.7950	0.7905
Qwen2.5-32B	+0.0096	[0.0027, 0.0165]	0.7887	0.7791
Llama-3.3-70B	+0.0246	[0.0159, 0.0332]	0.7826	0.7580

Table 5: Comparison of instruction-conditioned (O2) and state-aware performance (O3) across the model

Condition	Qwen2.5-7B	Phi-4-14B	Qwen2.5-32B	Llama-3.3-70B
Active operator	0.5608	0.5610	0.5589	0.5563
Operator zeroed	0.5298	0.5298	0.5259	0.5244
No instruction	0.5277	0.5264	0.5236	0.5231
Prompting	0.5517	0.5490	0.5466	0.5488

Table 6: Accuracy under operator and instruction ablations across the four models. The active operator achieves the highest accuracy for every model, while zeroing the operator or removing the intervention instruction reduces performance.

Model	WT Δ Bal	WT Δ TS	IV Δ Bal	IV Δ TS
Qwen2.5-7B	0.0000	0.0000	0.0000	0.0000
Phi-4-14B	-0.0051	-0.0326	-0.0056	-0.0667
Qwen2.5-32B	-0.0005	-0.0094	-0.0013	-0.0189
Llama-3.3-70B	+0.0000	-0.0008	-0.0001	-0.0017

Table 7: Sensitivity to incorrect intervention specifications. WT denotes the wrong-target condition, IV denotes the inverted-value condition, Δ Bal denotes the change in balanced score relative to the correctly specified intervention, and Δ TS denotes the corresponding change in target success. Larger negative values indicate greater sensitivity to the corrupted intervention.

Model	RI	NP	SI	DN	RInit	DE
Qwen2.5-7B	0.5779	0.5420	0.4845	0.7500	0.6260	0.6534
Phi-4-14B	0.5728	0.5415	0.4933	0.7500	0.6260	0.6534
Qwen2.5-32B	0.5965	0.5554	0.4954	0.7500	0.6260	0.6534
Llama-3.3-70B	0.5578	0.5362	0.5044	0.7500	0.6260	0.6534

Table 8: Average accuracy scores for the controls. RI denotes the real-intervention condition, NP the noncausal placebo, SI the shuffled-instruction condition, DN the do-nothing floor, RInit the random-initialized floor, and DE the do-everything floor.

decision under either condition. For the other models, the changes are small. Correctly identifying the target therefore does not by itself show that changing that target controls the final answer.

5.2.4 Real, Placebo, Shuffled, and Trivial Interventions

Real intervention conditions (Table 8) perform better than placebo and shuffled instructions on every model. An operator that makes little or no change can receive a high score. These values therefore need to be considered together with change accuracy and task correctness.

5.2.5 Change and Preservation

Table 9 shows that in the two-intervention setting, change in accuracy is about 0.54 to 0.57, and preservation is about 0.58 to 0.62. The do-nothing condition increases preservation while reducing change. An intervention should make the requested change while preserving unrelated information, as measuring just change is not sufficient.

5.3 Experiment 3: Multiple Interventions

We test interventions on multiple variables and repeated interventions on the same variable. For the repeated-intervention test, the model first sets a variable to one value and then sets the same variable to another. The final state should match the second intervention. From Table 10, the models often predict individual states correctly, but exact

accuracy for the full sequence ranges from only 3.4% to 7.2%. The models therefore have difficulty replacing an earlier intervention with a later intervention on the same variable.

5.4 Experiment 4: Generalization

Paraphrase accuracy remains between 0.826 and 0.843 across the four models. Accuracy on selecting the causal question is approximately 1.000. Evidence Inference accuracy ranges from 0.704 to 0.726. On one environment test, both the comparison method and the state-aware operator score 1.000 for every model and source. On the causal-ladder test, Qwen2.5-7B and Qwen2.5-32B show differences of +0.0007 and +0.0021, while Phi-4 and Llama-3.3-70B already score 1.000; as a result, they provide little information about differences between the methods.

5.4.1 External Causal Tasks

The external test uses a trained BERT-base model as the comparison method, which does not use O3, as shown in Table 11. Each run contains 178 and 39 items from CounterBench and CRASS, respectively.

The state-aware operator reaches 1.000 on CRASS for every model, but CRASS contains only 39 items per seed. CounterBench contains more items and shows larger differences for three of the

Model	CC	CP	CB	DN-C	DN-P	DN-B
Qwen2.5-7B	0.5402	0.6152	0.5777	0.4403	0.6265	0.5334
Phi-4-14B	0.5739	0.5753	0.5746	0.4604	0.5795	0.5200
Qwen2.5-32B	0.5586	0.5911	0.5748	0.4532	0.5871	0.5202
Llama-3.3-70B	0.5565	0.5849	0.5707	0.4533	0.5904	0.5219

Table 9: Change, preservation, and balanced scores under the clean intervention and do-nothing conditions. CC, CP, and CB denote clean change, clean preservation, and clean balanced score, respectively. DN-C, DN-P, and DN-B denote do-nothing change, do-nothing preservation, and do-nothing balanced score.

Model	Individual-state acc.	Exact overwrite-sequence acc.
Qwen2.5-7B	0.7066	0.0523
Phi-4-14B	0.7227	0.0718
Qwen2.5-32B	0.6748	0.0342
Llama-3.3-70B	0.7117	0.0668

Table 10: Accuracy on individual edited states and exact same-target overwrite sequences. Although the models often predict individual states correctly, their low exact sequence accuracy shows that they do not reliably execute successive interventions in which a later assignment must overwrite an earlier assignment to the same target.

four models. The difference between the comparison method and the state-aware operator becomes smaller for the larger models. However, since the models differ in more than size, we can only infer a correlation.

6 Discussion

No operator variant dominates across settings, since *O3* underperforms *O2* on F2, target pointer randomization leaves downstream outputs unchanged on three of four backbones, and exact sequence accuracy under repeated overwrites is at most 7.2%. Low residuals alone do not indicate valid interventions, as editors that make no change and editors that make random changes also attain zero route residual while their task accuracy stays at chance level. We therefore report each law residual alongside a task correctness gate and treat the algebraic laws as diagnostics rather than certificates of validity.

Explicit supervision reduces the identity, idempotence, and disjoint commutation residuals by 73.8% to 88.0% while changing balanced task accuracy by only 0.0035, or 0.35 percentage points. Ablations that omit one law loss at a time restore the corresponding residual. In contrast, the selective invariance residual does not decrease under its dedicated preservation objective.

The normalized distance $d(a, b)$ lies in $[0, 2)$, but no fixed threshold within that range separates causal from non-causal behavior. We therefore interpret residuals comparatively through paired reductions with confidence intervals, an executable oracle that attains zero residual at full task correctness on XOR and CCR.GB, and correctness controls against trivial baselines. This design of-

Source	Model	Comparison	O3	Difference
CounterBench	Qwen2.5-7B	0.6829	0.8062	+0.1233
CounterBench	Phi-4-14B	0.6952	0.7972	+0.1020
CounterBench	Qwen2.5-32B	0.7817	0.8396	+0.0579
CounterBench	Llama-3.3-70B	0.7483	0.7596	+0.0112
CRASS	Qwen2.5-7B	0.9705	1.0000	+0.0295
CRASS	Phi-4-14B	0.9487	1.0000	+0.0513
CRASS	Qwen2.5-32B	0.9615	1.0000	+0.0385
CRASS	Llama-3.3-70B	0.9436	1.0000	+0.0564

Table 11: Performance of the *O3* operator relative to the comparison condition across different models. Difference is calculated as *O3* minus the comparison score. All confidence intervals exclude zero, indicating a positive *O3* improvement for every evaluated backbone.

fers no support for claims that a residual below 0.1 implies causality.

On T5, the operator improves test accuracy by 0.0194 to 0.1065, equivalent to 1.94 to 10.65 percentage points, across all four backbones under locked testing despite nonzero route residuals. This shows that downstream utility and intervention validity are separable and should be reported separately. Describing a method as an intervention operator rather than a state perturbation commits it to testable behaviour under repetition, composition, and selective preservation, and CORE measures these properties directly.

7 Conclusion

We asked when a latent representation edit should be treated as a causal intervention rather than merely a successful perturbation. Across 23 registered experimental questions/analyses and four decoder backbones, the answer is not a single score. Explicit route-law supervision makes the learned transformation substantially more regular under identity, idempotence, commutation, and overwrite tests, while selective invariance remains unresolved under the current objective; trivial editors can also achieve perfect route residuals. State-aware *O3* is useful on external held-out causal tasks yet does not uniformly beat simpler operators and still fails same-target sequence semantics. The resulting standard is therefore three-dimensional: *task success*, *structural regularity*, and *correctness of the intervened world must be evaluated separately*. In direct answer to the title, a latent edit behaves like a causal intervention only when it changes the right decisions/world, obeys the relevant intervention laws, and outperforms trivial editors that can satisfy those laws vacuously. The present operators satisfy parts of that standard but not all of it, especially selective invariance and exact overwrite semantics. This separation turns “intervention” from a metaphor into a falsifiable claim about representation editing.

8 Limitations

The structural properties are diagnostics, not sufficient conditions for proving an SCM-level realization. The four-law audit is now complete, but selective invariance is a negative result: its preservation/change-sensitive score does not improve under the current dedicated objective, so we make no claim that CORE has solved all four laws. Some transfer tasks saturate; graph quality varies across sources; model architecture and scale are confounded; and A1 addressing claims remain validation/mechanism results. The framework currently covers hard interventions only. Exact same-target performance is poor, so the learned operators are not complete implementations of intervention algebra. The T3 pointer localizes targets but fails random-address, wrong-target, and inverted-value mechanism tests on three of four backbones; no general causal-path claim is made for pointer addressing. The L3 method labels include rounded numerical ties and shared evaluation lineage, so they are not treated as independent confirmations. Finally, the 39-item CRASS test subset is too small and too saturated to support a stand-alone scaling claim.

References

- Simon Bing, Jonas Wahl, Urmi Ninad, and Jakob Runge. 2023. Invariance & causal representation learning: Prospects and limitations. *arXiv preprint arXiv:2312.03580*.
- Stephan Bongers, Patrick Forré, Jonas Peters, and Joris M Mooij. 2021. Foundations of structural causal models with cycles and latent variables. *The Annals of Statistics*, 49(5):2885–2915.
- Yuefei Chen, Vivek K. Singh, Jing Ma, and Ruixiang Tang. 2026. [CounterBench: Evaluating and improving counterfactual reasoning in large language models](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 30350–30358.
- Jay DeYoung, Eric Lehman, Benjamin Nye, Iain Marshall, and Byron C. Wallace. 2020. Evidence inference 2.0: More data, better models. In *Proceedings of the 19th SIGBioMed Workshop on Biomedical Language Processing*, pages 123–132.
- Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, and 1 others. 2023. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature machine intelligence*, 5(3):220–235.

- Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. 2021. Causal abstractions of neural networks. In *Advances in Neural Information Processing Systems*, volume 34.
- John Gkountouras, Matthias Lindemann, Phillip Lippe, Efstratios Gavves, and Ivan Titov. 2025. Language agents meet causality—bridging llms and causal world models. In *International Conference on Learning Representations*, volume 2025, pages 37311–37336.
- Zihan Gu, Tianyi Zhang, Xinyan Zhang, Zhiyuan Wang, Han Zhang, Yuhao Wei, Jiacheng Lu, Tianyi Ma, Xingsheng Zhang, Hua Zhang, and 1 others. 2026. Diagnosing hidden instabilities in model editing via uncertainty quantification. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32544–32566.
- Wes Gurnee and Max Tegmark. 2024. Language models represent space and time. In *International Conference on Learning Representations*, volume 2024, pages 2483–2503.
- John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of EMNLP-IJCNLP*, pages 2733–2743.
- Zhijing Jin, Yuen Chen, Felix Leeb, Luigi Gresele, Ojasv Kamal, Zhiheng Lyu, Kevin Blin, Fernando Gonzalez Adauto, Max Kleiman-Weiner, Mrinmaya Sachan, and Bernhard Schölkopf. 2023. CLadder: Assessing causal reasoning in language models. In *Advances in Neural Information Processing Systems*.
- Arinbjörn Kolbeinsson, Kyle O’Brien, Tianjin Huang, Shanghua Gao, Shiwei Liu, Jonathan Schwarz, Anurag Vaidya, Faisal Mahmood, Marinka Zitnik, Tianlong Chen, and 1 others. 2025. Composable interventions for language models. In *International Conference on Learning Representations*, volume 2025, pages 50949–50982.
- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys*, 55(9):1–35.
- Wei Liu, Haomei Xu, Hongkai Liu, Zhiying Deng, Ruixuan Li, Heng Huang, Yee Whye Teh, and Wee Sun Lee. 2026a. Are we evaluating the edit locality of llm model editing properly? *arXiv preprint arXiv:2601.17343*.
- Yao-Yang Liu, Zhen Zheng, Feng Zhang, Jin-Cheng Feng, Yi-Yang Fu, Ji-Dong Zhai, Bing-Sheng He, Xiao Zhang, and Xiao-Yong Du. 2026b. A comprehensive taxonomy of prompt engineering techniques for large language models. *Frontiers of Computer Science*, 20(3):2003601.

Jacqueline R. M. A. Maasch, Alihan Hüyük, Xinnuo Xu, Aditya V. Nori, and Javier Gonzalez. 2025. Compositional causal reasoning evaluation in language models. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 42325–42365.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372.

Judea Pearl and 1 others. 2003. Causality: models, reasoning, and inference. *Econometric Theory*, 19(675–685):46.

Paul K Rubenstein, Sebastian Weichwald, Stephan Bongers, Joris M Mooij, Dominik Janzing, Moritz Grosse-Wentrup, and Bernhard Schölkopf. 2017. Causal consistency of structural equation models. *arXiv preprint arXiv:1707.00819*.

Alessandro Stolfo, Vidhisha Balachandran, Safoora Yousefi, Eric Horvitz, and Besmira Nushi. 2025. Improving instruction-following in language models through activation steering. In *International Conference on Learning Representations*, volume 2025, pages 55790–55823.

Nishant Subramani, Nivedita Suresh, and Matthew E Peters. 2022. Extracting latent steering vectors from pretrained language models. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 566–581.

Niket Tandon, Bhavana Dalvi, Keisuke Sakaguchi, Peter Clark, and Antoine Bosselut. 2019. WIQA: A dataset for “what if...” reasoning over procedural text. In *Proceedings of EMNLP-IJCNLP*, pages 6076–6085.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Ulisse Mini, and Monte MacDiarmid. 2024. Activation addition: Steering language models without optimization.

Luping Wang, Sheng Chen, Linnan Jiang, Shu Pan, Runze Cai, Sen Yang, and Fei Yang. 2025. Parameter-efficient fine-tuning in large language models: a survey of methodologies. *Artificial Intelligence Review*, 58(8):227.

Zhengxuan Wu, Karel D’Oosterlinck, Atticus Geiger, Amir Zur, and Christopher Potts. 2023. Causal proxy models for concept-based model explanations. In *Proceedings of ICML*, pages 37313–37334.

Kai Xiong, Xiao Ding, Yixin Cao, Yuxiong Yan, Li Du, Yufei Zhang, Jinglong Gao, Jiaqian Liu, Bing Qin, and Ting Liu. 2025. Com²: A causal-guided benchmark for exploring complex commonsense reasoning in large language models. In *Proceedings of ACL*, pages 16119–16140.

Wei Jie Yeo, Ranjan Satapathy, and Erik Cambria. 2025. Towards faithful natural language explanations: A study using activation patching in large language models. In *Proceedings of the 2025 Conference on*

Empirical Methods in Natural Language Processing, pages 10436–10458.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, and 1 others. 2023. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*.

A Exact Loss and Measurement Definitions

Let $d(a, b) = \|a - b\|_2^2 / (\|a\|_2^2 + \|b\|_2^2 + \epsilon)$. The implementation uses this normalized representation distance for route-based residuals. Since $\|a - b\|^2 \leq 2(\|a\|^2 + \|b\|^2)$, $0 \leq d < 2$ (up to the stabilizing ϵ). Exact equality gives 0; equal-norm orthogonal states give approximately 1; equal-norm opposite states approach 2. This is why residuals slightly above 1 are valid. A mathematically exact oracle that maps two semantically equivalent routes to the same intervened representation has residual 0, but 0 is not sufficient for causal validity because do-nothing can also achieve it.

Intervention matching. For factual input x and explicitly intervened realization $x^{\text{do}(e)}$,

$$\mathcal{L}_{\text{match}} = d(T_e(h(x)), h(x^{\text{do}(e)})). \quad (9)$$

Identity. $\mathcal{L}_{\text{id}} = d(T_{e_\emptyset}(h), h)$.

Idempotence. $\mathcal{L}_{\text{idem}} = d(T_e(T_e(h)), T_e(h))$.

Disjoint commutation. For disjoint a, b ,

$$\mathcal{L}_{\text{comm}} = d(T_b(T_a(h)), T_a(T_b(h))). \quad (10)$$

Selective invariance. The behavioral form penalizes incorrect changes on graph-derived non-descendants while separately scoring descendant sensitivity. We never report preservation without a change-sensitive metric, because do-nothing is otherwise optimal.

Last-write diagnostic. For two writes to the same target, the residual compares the sequential state with the state from the final write alone. This is a diagnostic rather than one of the four counted laws.

B Addressing Architecture and Mechanical Checks

T2 uses a masked encode-only design. Intervention tokens may encode the passage before the edit, but

world-state slots cannot attend to intervention tokens before editing, and intervention tokens are removed from downstream layers after the edit point. The mechanical check is that zeroing the editor must collapse toward the no-instruction condition; otherwise T2 degenerates into ordinary prompting.

T3 pools the target span to an address vector and attends over latent slot keys. Gold/string-derived spans are used for the primary experiment so that target finding and target-conditioned editing are not confounded. The mechanism check randomizes the address while preserving the rest of the input. The observed near-invariance of downstream score under this randomization is why the paper separates pointer localization from pointer necessity.

C Registered Experiment Index and Primary Outcomes

This appendix maps each registry ID to the primary scalar actually reported in the manuscript. Rows that reuse shared cells are marked explicitly and must not be read as independent replications.

D Detailed Law-Dose Results

D.1 L1: one, three, and four laws

Each row below aggregates 20 seeds $\times$ 4 backbones ($n = 80$). Values are mean [95% Student- t CI].

Paired Base $\rightarrow$ Full effects ($n = 80$). Balanced -0.00356 $[-0.00513, -0.00199]$; identity -0.00293 $[-0.00340, -0.00247]$; idempotence -0.55005 $[-0.59702, -0.50309]$; commutation -0.14135 $[-0.16937, -0.11333]$; last-write -0.62793 $[-0.69446, -0.56140]$.

D.2 L2: leave one law out

Each row aggregates $n = 80$ model-seed cells.

Paired own-law effects. Dropping identity increases identity by $+0.11969$ $[.11044, .12893]$; dropping idempotence increases idempotence by $+0.18069$ $[.17230, .18907]$; dropping commutation increases commutation by $+0.19326$ $[.17889, .20762]$. Balanced-accuracy changes are $+0.00015$ $[-0.00096, .00125]$, -0.00066 $[-.00185, .00053]$, and $+0.00259$ $[.00130, .00389]$, respectively.

D.3 L3: common battery and checkpoint aliasing

L3 has five seeds per method per backbone ($n = 20$ macro cells per label). The artifact hashes show that matched LoRA, LoReFT, and O2 share identical checkpoints within each model/seed; O1 and task-vector-add likewise share checkpoints. The equal rows are therefore aliases in this battery, not independent implementations.

E Selective-Invariance Detail

The selective-invariance audit contains 1,640 checkpoint/method/seed rows spanning BERT-base and the four decoder backbones. The score is the balanced mean of descendant/change sensitivity and non-descendant preservation. This pairing prevents a do-nothing editor from being rewarded for preservation alone. In L1, decoder CORE-Base/Core-I/CORE-Full scores are close within each backbone; in L2, removing the selective-invariance objective does not reliably decrease the metric. On BERT-base, Full versus $-SI$ is .49383 versus .49377 on XOR and .51356 versus .51356 on CLadder. We therefore report selective invariance as measured but unsupported by the current dedicated loss.

F Oracle Calibration

On frozen validation artifacts, an executable ground-truth intervention oracle yields identity, idempotence, and commutation residual exactly 0 with exact correctness 1.000 on both XOR and CCR.GB. CCR.GB includes 600 identity, 1,800 idempotence, and 2,974 commutation evaluations. This validates the zero-residual endpoint of the measurement code while also illustrating why zero alone is insufficient: do-nothing can share the same structural endpoint without causal correctness.

G Control and Diagnostic Detail

G.1 F1 change/preservation decomposition

F1 is decomposed into change and preservation so that a high preservation score cannot mask failure to update descendants.

G.2 A2 editor-necessity controls

A2 tests whether the learned editor contributes beyond a zeroed editor, no instruction, and ordinary prompting.

ID	Primary outcome	Q7B	P14B	Q32B	L70B
F1	clean two-edit balanced	.5777	.5746	.5748	.5707
F2	O2—O3 clean contrast	+.0277	+.0045	+.0096	+.0246
F3	clean balanced intervention	.5557	.5536	.5633	.5471
F4	reader-scale clean balanced	.5684	.5594	.5641	.5545
A1	addressing balanced score	.5457	.5451	.5432	.5388
A2	active-editor balanced score	.5608	.5610	.5589	.5563
A3	correct-address balanced score	.5298	.5299	.5255	.5250
L1	law-variant balanced aggregate	.5788	.5845	.5785	.5771
L2	leave-one-law balanced aggregate	.6157	.6278	.6224	.6188
L3	eight-method law-suite aggregate	.5566	.5465	.5521	.5532
C1	real intervention condition	.5779	.5728	.5965	.5578
C2	trained O3 structural aggregate	.5779	.5728	.5965	.5578
C3	operator-necessity/derived contrast	.2348	.0045	.0096	.0246
C4	parameter-accounting contrast	—	.0045	.0096	.0246
T1	composition contrast (O2—O3)	.0277	.0045	.0096	.0246
T2	validation task accuracy	.9479	.9489	.9488	.9442
T3	paraphrase robustness	.8311	.8364	.8263	.8432
T4	rung-transfer validation accuracy	.9164	.9218	.9172	.9200
T5 BERT-base scorer	external validation accuracy	.4748	.5549	.5567	.3664
G1	estimand-question accuracy	1.0000	1.0000	.9999	1.0000
G2	exact shared-target sequence	.0523	.0718	.0342	.0668
G3	configuration: selected edit-layer score	.5941	.5925	.5885	.5921
G4	lower-rung Evidence Inference	.7263	.7036	.7117	.7191

Table 12: Primary scalar recorded for every registered question/model. Metrics differ by experiment and are not a single common scale. F2/T1 and parts of C3/C4 are views of shared underlying cells; C1/C2 also share trained O3 cells. G3 is configuration selection, not a scientific outcome.

Shared contrasts. Several IDs intentionally consume the same underlying trained checkpoints or derived contrast. In particular, repeated numerical values across F2/T1 and parts of C3/C4 are disclosed rather than interpreted as independent replications. The registry counts questions/analyses, not independent training pipelines.

Variant	Balanced	Identity	Idempotence	Comm.	Last-write
Base	.6220 [.6072,.6369]	.0037 [.0031,.0042]	.6248 [.5758,.6738]	.1916 [.1612,.2220]	.7756 [.7036,.8476]
I	.6222 [.6075,.6369]	.0015 [.0012,.0017]	.1039 [.0913,.1165]	.0840 [.0780,.0900]	.2468 [.2298,.2638]
Full	.6185 [.6042,.6327]	.0007 [.0006,.0008]	.0747 [.0676,.0819]	.0502 [.0471,.0533]	.1477 [.1401,.1553]
Do-nothing	.5349 [.5306,.5391]	0	0	0	0
Random-init	.5341 [.5299,.5383]	0	0	0	0
Do-everything	.5466 [.5390,.5543]	.9982 [.9938,1.0027]	1.0000 [.9995,1.0006]	0	1.0000 [.9995,1.0006]

Table 13: L1 uncertainty-complete route-law macro table. Selective invariance is reported separately because it is a preservation/change-sensitive behavioral score rather than a route distance.

G.3 C1 and C2

C1 tests causal specificity against placebo and shuffled conditions; C2 places the trained editor beside degenerate structural floors.

G.4 A3 wrong-target and inverted-value controls

A3 perturbs the address and command value on frozen checkpoints to test whether the nominal addressing pathway is decision-relevant.

G.5 G2 shared-target sequence detail

G2 distinguishes bit-level correctness from exact same-target overwrite semantics.

H Data-Split Audit

All 23 experiments have the required data resources; “not required” denotes protocols that do not use that split rather than missing data. Selected row counts are: F1 4,000 train/1,200 validation; F2 54,282/9,567; F3 48,000/14,400; F4

4,000/1,200; A1 58,282/10,767 plus 6,788 reserved test rows; L1/L2 5,109/1,347; L3 4,000/8,400; C1 72,000/21,600; T2 11,000/2,000/2,000; T4 33,000/6,000/6,000; and T5 1,476 validation plus 590 test rows. Test bundles were copied and locked before scoring.

I Locked-Test Source Breakdown

The T5 locked artifact reports 39 CRASS items and 178 CounterBench items per seed (217 evaluable items; 477 raw test rows before source exclusions, with Com² excluded). The overall T5 score exactly matches the item-weighted micro-average of the two source accuracies, not their unweighted mean. T2 is 1.000 for both baseline and O3 on both recorded sources (`weak_arrows` and `weak_arrows_binary_t`) for all four models. T4 is likewise saturated on `changing_only`; on `imagining_only` and `joint`, only the Qwen backbones leave tiny baseline headroom, which O3 closes.

Variant	Balanced	Identity	Idempotence	Comm.	Last-write
Full	.6201 [.6054,.6347]	.0008 [.0006,.0009]	.0757 [.0683,.0832]	.0503 [.0474,.0533]	.1501 [.1423,.1579]
– identity	.6202 [.6055,.6349]	.1205 [.1112,.1297]	.0763 [.0688,.0838]	.0504 [.0475,.0534]	.1507 [.1430,.1585]
– idempotence	.6194 [.6045,.6343]	.0014 [.0012,.0017]	.2564 [.2440,.2688]	.0158 [.0144,.0171]	.2668 [.2540,.2797]
– commutation	.6227 [.6077,.6376]	.0007 [.0006,.0009]	.0039 [.0033,.0046]	.2436 [.2283,.2589]	.2803 [.2624,.2981]
– selective invariance	.6235 [.6085,.6386]	.0015 [.0012,.0018]	.1071 [.0936,.1206]	.0842 [.0786,.0898]	.2526 [.2350,.2703]

Table 14: L2 route-law macro means and 95% intervals. Selective invariance is measured separately; the – selective-invariance row is retained here to show cross-objective effects on the route metrics.

Label/class	Balanced	Identity	Idempotence	Comm.	Last-write
Prompting	.5370 [.5278,.5461]	0	0	0	0
LoRA=LoReFT=O2	.5510 [.5378,.5642]	.0088 [.0055,.0121]	.0251 [.0174,.0328]	0	.0251 [.0174,.0328]
O1=task-vector	.5438 [.5289,.5586]	1.0040 [.9936,1.0143]	.9919 [.9853,.9985]	0	.9919 [.9853,.9985]
Router	.5420 [.5278,.5562]	.9048 [.7596,1.0499]	.8907 [.7480,1.0334]	0	.8907 [.7480,1.0334]
O3	.5972 [.5626,.6318]	.7016 [.5687,.8345]	.7698 [.6381,.9015]	.1954 [.1327,.2580]	.9198 [.7507,1.0888]

Table 15: L3 uncertainty-complete macro table. Exact-equality labels are collapsed because their checkpoint SHA-256 hashes are identical in this artifact.

	Q7B	P14B	Q32B	L70B
Clean change	.5402	.5739	.5586	.5565
Clean preservation	.6152	.5753	.5911	.5849
Clean balanced	.5777	.5746	.5748	.5707
Do-nothing change	.4403	.4604	.4532	.4533
Do-nothing preservation	.6265	.5795	.5871	.5904
Do-nothing balanced	.5334	.5200	.5202	.5219

Table 16: F1 shows why preservation alone is insufficient.

Condition	Q7B	P14B	Q32B	L70B
Active editor	.5608	.5610	.5589	.5563
Editor zeroed	.5298	.5298	.5259	.5244
No instruction	.5277	.5264	.5236	.5231
Prompting	.5517	.5490	.5466	.5488

Table 17: A2 balanced intervention scores.

Condition	Q7B	P14B	Q32B	L70B
Real	.5779	.5728	.5965	.5578
Noncausal placebo	.5420	.5415	.5554	.5362
Shuffled	.4845	.4933	.4954	.5044
Do-nothing floor	.7500	.7500	.7500	.7500
Random-init floor	.6260	.6260	.6260	.6260
Do-everything floor	.6534	.6534	.6534	.6534
Trained O3	.5779	.5728	.5965	.5578

Table 18: C1 causal-specificity and C2 degenerate-floor aggregates. The C2 metric is intentionally shown as a warning: trivial methods can score highly.

J Training Configuration Fields

The configuration audit binds recipes to registry/runner sources rather than inferring them from result tables. Table 22 summarizes the recoverable settings. The “BERT-base runner/scorer labels name the archived training code path from which these hyperparameters are recoverable; they are not names for the Qwen/Phi/Llama decoder backbones reported in the four-backbone result tables. Decoder-expansion settings that were not independently serialized are not inferred. L1/L2 use AdamW, 2×10^{-5} learning rate, 3 epochs, effective batch 64 over four GPUs, rank-16 operators at layer 4, bf16, and maximum validation macro-F1 selection. F2 uses AdamW at 10^{-3} for 500 steps, effective batch 64, rank-16 O2/O3 at layer 4 over 16 appended state tokens; matched LoRA uses rank 34. L3 uses AdamW at 5×10^{-3} for 500 steps. T2/T4/T5 use AdamW at 3×10^{-5} for 3 epochs, effective batch 32, rank 16, layer 4, with law-loss weight 0.1 for T2/T4.

For locked tests, T2 has no separately named baseline arm and T4 consists of three registered O3 transfer arms; these should not be described as generic baseline comparisons. T5 baseline is the trained BERT-base candidate scorer with learned state tokens/head that bypasses O3.

K Compute, Licensing, and Resource Use

The experiment registry was designed around staged rather than full-factorial evaluation. The original compute plan budgeted four H100 GPUs for a 28-day window and approximately 1,791 GPU-hours including debugging/re-runs, with screening on smaller/cheaper configurations before

Backbone	Wrong-target Δ bal.	Wrong-target Δ target	Inverted-value Δ bal.	Inverted-value Δ target
Qwen2.5-7B	.0000	.0000	.0000	.0000
Phi-4-14B	-.0051	-.0326	-.0056	-.0667
Qwen2.5-32B	-.0005	-.0094	-.0013	-.0189
Llama-3.3-70B	+.0000	-.0008	-.0001	-.0017

Table 19: A3 positive-control perturbations, perturbation minus correct-address mean over $n = 20$ seeds. Only Phi-4 shows resolved target-success degradation for both perturbations; Qwen-7B is exactly decision-inert.

	Q7B	P14B	Q32B	L70B
Bit accuracy	.7066	.7227	.6748	.7117
Exact sequence	.0523	.0718	.0342	.0668

Table 20: G2: local correctness is much higher than exact overwrite-sequence correctness.

Source	Model	Baseline	O3	Δ
CounterBench	Q7B	.6829	.8062	+.1233
	P14B	.6952	.7972	+.1020
	Q32B	.7817	.8396	+.0579
	L70B	.7483	.7596	+.0112
CRASS	Q7B	.9705	1.0000	+.0295
	P14B	.9487	1.0000	+.0513
	Q32B	.9615	1.0000	+.0385
	L70B	.9436	1.0000	+.0564

Table 21: T5 locked-test source-level results. Each seed contains 178 CounterBench and 39 CRASS evaluable items (217 total); the overall T5 score is their item-weighted micro-average. CRASS therefore contributes 18.0% of the aggregate.

decoder confirmation. The final four-backbone run artifacts record per-cell GPU counts but do not yet provide a single reconciled realized GPU-hour total; we therefore report the planned budget as design information and do not present it as realized usage. The current export is insufficient to reconstruct a trustworthy total without double-counting shared cells.

Dataset/software licensing was tracked source by source. CLadder is MIT; WIQA is Apache-2.0; CCR.GB code is GPL-3.0 and is used via clone-and-run rather than vendored redistribution; CRASS is Apache-2.0; Com² is consumed from the authors’ release without redistributing their repository content in CORE. Where a source has no explicit license, the paper does not claim redistribution rights. The release contains conversion/fetch scripts and derived metadata rather than third-party source code or raw data whose licenses prohibit redistribution.

Block	LR	Epochs/steps	Eff. batch	Rank	Edit layer/position	Precision	Selection
L1/L2 BERT-base runner	$2e-5$	3 ep	64	16	layer 4 / 30 world slots	bf16	max val macro-F1
F2 BERT-base runner	$1e-3$	500 st	64	16	layer 4 / 16 state tokens	fp32	fixed protocol
L3 BERT-base runner	$5e-3$	500 st	64	16	layer 4 / 16 state tokens	fp32	fixed protocol
T2/T4 BERT-base runner	$3e-5$	3 ep	32	16	layer 4	recorded default	registered
T5 BERT-base scorer	$3e-5$	3 ep	32	16	layer 4 / 16 state tokens	fp32	max val group-macro acc.

Table 22: Source-bound configuration summary from the archived training configuration export. “BERT-base runner/scorer denotes the archived component whose optimizer settings are recorded, not the decoder-backbone identity used in the four-backbone expansion. Trainable parameter counts were not separately serialized for every block and are therefore not invented. F2 records 234,240 O2 and 1,242,624 O3 operator/adaptor parameters; matched LoRA has 1,253,376 trainable parameters.